

实验报告

张晨轩 2024K8009929011

1 互联网协议分析实验

1.1 实验内容

本实验使用mininet搭建网络环境，在节点h1上启动wireshark对 h1-eth0 网卡进行抓包，并使用 `wget` `www.ucas.ac.cn` 的页面。

通过分析wireshark捕获的数据包，观察ARP、DNS、TCP、HTTP/HTTPS等协议在网页获取过程中的作用，同时观察不同协议之间的封装关系，并分析h1获取页面时各协议的运行过程。

1.2 实验流程

- 启动带NAT功能的mininet网络 `sudo mn --nat`
- 在mininet中打开h1的终端 `mininet> xterm h1`
- 在h1中配置DNS服务器 `echo "nameserver 8.8.8.8" > /etc/resolv.conf`
- 在h1中启动wireshark `wireshark &`
- 在wireshark中选择 h1-eth0 网卡开始抓包
- 使用以下命令访问 UCAS 网站 `wget -U "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/123.0 Safari/537.36" https://www.ucas.ac.cn`

网页访问结束后停止wireshark抓包，并分别对ARP、DNS、TCP和HTTPS数据包进行分析。

1.3 实验结果及分析

1.3.1 ARP协议分析

在wireshark中使用 `arp` 作为过滤条件

可以观察到h1发出的ARP请求：

```
1 | who has 10.0.0.3? Tell 10.0.0.1
```

随后收到对应的ARP响应：

```
1 | 10.0.0.3 is at ce:73:a6:35:88:a5
```

由抓包结果可知，h1的IP地址为 `10.0.0.1`，需要将数据发送到下一跳 `10.0.0.3`。由于数据链路层发送 Ethernet 帧时需要知道下一跳设备的MAC地址，因此h1首先使用ARP查询 `10.0.0.3` 对应的MAC地址。ARP响应表明该地址对应的MAC地址为：

```
1 | ce:73:a6:35:88:a5
```

因此，本次实验中ARP完成了下一跳IP地址到MAC地址的映射。

ucas_ac_cn.pcapng						
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help						
arp						
No.	Time	Source	Destination	Protocol	Length	Info
3	18.797812233	ae:fa:e2:66:a3:ba	Broadcast	ARP	42	Who has 10.0.0.3? Tell 10.0.0.1
4	18.798078533	ce:73:a6:35:88:a5	ae:fa:e2:66:a3:ba	ARP	42	10.0.0.3 is at ce:73:a6:35:88:a5
45	29.207955905	ce:73:a6:35:88:a5	ae:fa:e2:66:a3:ba	ARP	42	Who has 10.0.0.1? Tell 10.0.0.3
46	29.207965571	ae:fa:e2:66:a3:ba	ce:73:a6:35:88:a5	ARP	42	10.0.0.1 is at ae:fa:e2:66:a3:ba

1.3.2 DNS协议分析

在wireshark中使用 dns 作为过滤条件

可以观察到h1向DNS服务器 8.8.8.8 发出的查询请求。其中A记录查询内容为：

```

1 Name: www.ucas.ac.cn
2 Type: A
3 Class: IN

```

```

▶ Frame 5: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface h1-eth0, id 0
▶ Ethernet II, Src: ae:fa:e2:66:a3:ba (ae:fa:e2:66:a3:ba), Dst: ce:73:a6:35:88:a5 (ce:73:a6:35:88:a5)
▶ Internet Protocol Version 4, Src: 10.0.0.1, Dst: 8.8.8.8
▶ User Datagram Protocol, Src Port: 39525, Dst Port: 53
▼ Domain Name System (query)
  Transaction ID: 0xcfc6
  ▶ Flags: 0x0100 Standard query
  Questions: 1
  Answer RRs: 0
  Authority RRs: 0
  Additional RRs: 0
  ▼ Queries
    ▼ www.ucas.ac.cn: type A, class IN
      Name: www.ucas.ac.cn
      [Name Length: 14]
      [Label Count: 4]
      Type: A (1) (Host Address)
      Class: IN (0x0001)

```

DNS服务器随后返回响应，响应中包含：

```

1 Name: www.ucas.ac.cn
2 Type: A
3 Address: 124.16.77.5

```

因此，本次实验中：www.ucas.ac.cn 的IP地址为 124.16.77.5

```

▶ Frame 10: 90 bytes on wire (720 bits), 90 bytes captured (720 bits) on interface h1-eth0, id 0
▶ Ethernet II, Src: ce:73:a6:35:88:a5 (ce:73:a6:35:88:a5), Dst: ae:fa:e2:66:a3:ba (ae:fa:e2:66:a3:ba)
▶ Internet Protocol Version 4, Src: 8.8.8.8, Dst: 10.0.0.1
▶ User Datagram Protocol, Src Port: 53, Dst Port: 39525
▼ Domain Name System (response)
  Transaction ID: 0xcfc6
  ▶ Flags: 0x8180 Standard query response, No error
  Questions: 1
  Answer RRs: 1
  Authority RRs: 0
  Additional RRs: 0
  ▼ Queries
    ▼ www.ucas.ac.cn: type A, class IN
      Name: www.ucas.ac.cn
      [Name Length: 14]
      [Label Count: 4]
      Type: A (1) (Host Address)
      Class: IN (0x0001)
  ▼ Answers
    ▼ www.ucas.ac.cn: type A, class IN, addr 124.16.77.5
      Name: www.ucas.ac.cn
      Type: A (1) (Host Address)
      Class: IN (0x0001)
      Time to live: 3193 (53 minutes, 13 seconds)
      Data length: 4
      Address: 124.16.77.5
      [Request In: 7]
      [Time: 0.239727117 seconds]

```

该DNS请求的Transaction ID为 0xcfc6，其中包含以下内容：

1	Answer RRs: 1
2	Request In: 7
3	Time: 0.239727117 seconds

可得从对应请求到响应的时间约为0.24s

抓包过程中还观察到了AAAA类型的DNS查询。A记录用于查询域名对应的IPv4地址，AAAA记录用于查询IPv6地址。

DNS在本实验中的作用是将用户输入的域名 `www.ucas.ac.cn` 映射为网络层可以使用的IP地址 `124.16.77.5`。

1.3.3 TCP协议分析

DNS 解析完成后，h1开始与服务器 `124.16.77.5` 建立TCP连接。

本次连接的基本信息为：

1	客户端 IP: 10.0.0.1
2	客户端端口: 44642
3	服务器 IP: 124.16.77.5
4	服务器端口: 443
5	TCP Stream index: 0

wireshark中可以观察到TCP三次握手 `SYN -> SYN,ACK -> ACK`

其中 `10.0.0.1:44642` 发起 `SYN` 和 `ACK`，应答 `SYN,ACK`；`124.16.77.5:443` 发起 `SYN,ACK`，应答 `SYN` 和 `ACK`

三次握手对应时间如下：

```
1 | SYN: 24.042223435 s
2 | SYN, ACK: 24.043229145 s
3 | ACK: 24.043261499 s
```

从 SYN 到 SYN, ACK 的时间差约为**1.01ms**

11	24.042223435	10.0.0.1	124.16.77.5	TCP	74	44642 → 443	[SYN] Seq=0 Win=
12	24.043229145	124.16.77.5	10.0.0.1	TCP	74	443 → 44642	[SYN, ACK] Seq=0
13	24.043261499	10.0.0.1	124.16.77.5	TCP	66	44642 → 443	[ACK] Seq=1 Ack=

进一步查看第一个SYN包，可以看到：

```
1 | Source Port: 44642
2 | Destination Port: 443
3 | Sequence Number: 0
4 | Flags: SYN
```

```
▶ Frame 11: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface h1-eth0, id 0
▶ Ethernet II, Src: ae:fa:e2:66:a3:ba (ae:fa:e2:66:a3:ba), Dst: ce:73:a6:35:88:a5 (ce:73:a6:35:88:a5)
▶ Internet Protocol Version 4, Src: 10.0.0.1, Dst: 124.16.77.5
▼ Transmission Control Protocol, Src Port: 44642, Dst Port: 443, Seq: 0, Len: 0
  Source Port: 44642
  Destination Port: 443
  [Stream index: 0]
  ▶ [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 3837630873
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  1010 ... = Header Length: 40 bytes (10)
  ▶ Flags: 0x002 (SYN)
  Window: 42340
  [Calculated window size: 42340]
  Checksum: 0xd344 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  ▶ Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, No-Operation (NOP), Window scale
  ▶ [Timestamps]
```

TCP 三次握手的过程为：客户端首先发送SYN请求建立连接，服务器返回SYN和 ACK，表示同意建立连接并确认客户端的请求，客户端再发送 ACK 进行确认。完成三次握手后，双方建立TCP连接。

1.3.4 HTTPS协议分析

本实验访问的网址为：<https://www.ucas.ac.cn>，服务器端口为 443，因此TCP连接建立后继续进行TLS握手。

在wireshark中可以观察到TLS Client Hello数据包，其中包含：

```
1 | Handshake Protocol: Client Hello
2 | Server Name: www.ucas.ac.cn
```

Transport Layer Security

- TLV1.2 Record Layer: Handshake Protocol: Client Hello
 - Content Type: Handshake (22)
 - Version: TLS 1.0 (0x0301)
 - Length: 493
 - Handshake Protocol: Client Hello
 - Handshake Type: Client Hello (1)
 - Length: 399
 - Version: TLS 1.2 (0x0303)
 - Random: c978530785eb2631c8d5dd0ce06861069eeac2163140ebdefa22ee4086a0769
 - Session ID Length: 32
 - Session ID: 17a8343b1982dab0c2e9f92a79d39452b81e566e1cd42f761632e18749b203a
 - Cipher Suites Length: 150
 - Cipher Suites (75 suites)
 - Compression Methods Length: 1
 - Compression Methods (1 method)
 - Extensions Length: 176
 - Extension: server_name (len=19) name=www.ucas.ac.cn
 - Extension: ec_point_formats (len=4)
 - Extension: supported_groups (len=22)
 - Extension: session_ticket (len=0)
 - Extension: encrypt_then_mac (len=0)
 - Extension: extended_master_secret (len=0)
 - Extension: post_handshake_auth (len=0)
 - Extension: signature_algorithms (len=42)
 - Extension: supported_versions (len=5) TLS 1.3, TLS 1.2
 - Extension: psk_key_exchange_modes (len=2)
 - Extension: key_share (len=38) x25519
 - [JA4: t13d751100.479067518aa3.f08d57f048c1]
 - [JA4_r (truncated): t13d751100.902f.0032.0033.0035.0038.0039.003c.003d.0040.0041.0044.0045.0067.006a.006b.0064.0067.0068.006c.006d.006e.006f.0070.0071.0072.0073.0074.0075.0076.0077.0078.0079.0080.0081.0082.0083.0084.0085.0086.0087.0088.0089.0090.0091.0092.0093.0094.0095.0096.0097.0098.0099.0100.0101.0102.0103.0104.0105.0106.0107.0108.0109.0110.0111.0112.0113.0114.0115.0116.0117.0118.0119.0120.0121.0122.0123.0124.0125.0126.0127.0128.0129.0130.0131.0132.0133.0134.0135.0136.0137.0138.0139.0140.0141.0142.0143.0144.0145.0146.0147.0148.0149.0150.0151.0152.0153.0154.0155.0156.0157.0158.0159.0160.0161.0162.0163.0164.0165.0166.0167.0168.0169.0170.0171.0172.0173.0174.0175.0176.0177.0178.0179.0180.0181.0182.0183.0184.0185.0186.0187.0188.0189.0190.0191.0192.0193.0194.0195.0196.0197.0198.0199.0200.0201.0202.0203.0204.0205.0206.0207.0208.0209.0210.0211.0212.0213.0214.0215.0216.0217.0218.0219.0220.0221.0222.0223.0224.0225.0226.0227.0228.0229.0230.0231.0232.0233.0234.0235.0236.0237.0238.0239.0240.0241.0242.0243.0244.0245.0246.0247.0248.0249.0250.0251.0252.0253.0254.0255.0256.0257.0258.0259.0260.0261.0262.0263.0264.0265.0266.0267.0268.0269.0270.0271.0272.0273.0274.0275.0276.0277.0278.0279.0280.0281.0282.0283.0284.0285.0286.0287.0288.0289.0290.0291.0292.0293.0294.0295.0296.0297.0298.0299.0300.0301.0302.0303.0304.0305.0306.0307.0308.0309.0310.0311.0312.0313.0314.0315.0316.0317.0318.0319.0320.0321.0322.0323.0324.0325.0326.0327.0328.0329.0330.0331.0332.0333.0334.0335.0336.0337.0338.0339.0340.0341.0342.0343.0344.0345.0346.0347.0348.0349.0350.0351.0352.0353.0354.0355.0356.0357.0358.0359.0360.0361.0362.0363.0364.0365.0366.0367.0368.0369.0370.0371.0372.0373.0374.0375.0376.0377.0378.0379.0380.0381.0382.0383.0384.0385.0386.0387.0388.0389.0390.0391.0392.0393.0394.0395.0396.0397.0398.0399.0400.0401.0402.0403.0404.0405.0406.0407.0408.0409.0410.0411.0412.0413.0414.0415.0416.0417.0418.0419.0420.0421.0422.0423.0424.0425.0426.0427.0428.0429.0430.0431.0432.0433.0434.0435.0436.0437.0438.0439.0440.0441.0442.0443.0444.0445.0446.0447.0448.0449.0450.0451.0452.0453.0454.0455.0456.0457.0458.0459.0460.0461.0462.0463.0464.0465.0466.0467.0468.0469.0470.0471.0472.0473.0474.0475.0476.0477.0478.0479.0480.0481.0482.0483.0484.0485.0486.0487.0488.0489.0490.0491.0492.0493.0494.0495.0496.0497.0498.0499.0500.0501.0502.0503.0504.0505.0506.0507.0508.0509.0510.0511.0512.0513.0514.0515.0516.0517.0518.0519.0520.0521.0522.0523.0524.0525.0526.0527.0528.0529.0530.0531.0532.0533.0534.0535.0536.0537.0538.0539.0540.0541.0542.0543.0544.0545.0546.0547.0548.0549.0550.0551.0552.0553.0554.0555.0556.0557.0558.0559.0560.0561.0562.0563.0564.0565.0566.0567.0568.0569.0570.0571.0572.0573.0574.0575.0576.0577.0578.0579.0580.0581.0582.0583.0584.0585.0586.0587.0588.0589.0590.0591.0592.0593.0594.0595.0596.0597.0598.0599.0600.0601.0602.0603.0604.0605.0606.0607.0608.0609.0610.0611.0612.0613.0614.0615.0616.0617.0618.0619.0620.0621.0622.0623.0624.0625.0626.0627.0628.0629.0630.0631.0632.0633.0634.0635.0636.0637.0638.0639.0640.0641.0642.0643.0644.0645.0646.0647.0648.0649.0650.0651.0652.0653.0654.0655.0656.0657.0658.0659.0660.0661.0662.0663.0664.0665.0666.0667.0668.0669.0670.0671.0672.0673.0674.0675.0676.0677.0678.0679.0680.0681.0682.0683.0684.0685.0686.0687.0688.0689.0690.0691.0692.0693.0694.0695.0696.0697.0698.0699.0700.0701.0702.0703.0704.0705.0706.0707.0708.0709.0710.0711.0712.0713.0714.0715.0716.0717.0718.0719.0720.0721.0722.0723.0724.0725.0726.0727.0728.0729.0730.0731.0732.0733.0734.0735.0736.0737.0738.0739.0740.0741.0742.0743.0744.0745.0746.0747.0748.0749.0750.0751.0752.0753.0754.0755.0756.0757.0758.0759.0760.0761.0762.0763.0764.0765.0766.0767.0768.0769.0770.0771.0772.0773.0774.0775.0776.0777.0778.0779.0780.0781.0782.0783.0784.0785.0786.0787.0788.0789.0790.0791.0792.0793.0794.0795.0796.0797.0798.0799.0800.0801.0802.0803.0804.0805.0806.0807.0808.0809.0810.0811.0812.0813.0814.0815.0816.0817.0818.0819.0820.0821.0822.0823.0824.0825.0826.0827.0828.0829.0830.0831.0832.0833.0834.0835.0836.0837.0838.0839.0840.0841.0842.0843.0844.0845.0846.0847.0848.0849.0850.0851.0852.0853.0854.0855.0856.0857.0858.0859.0860.0861.0862.0863.0864.0865.0866.0867.0868.0869.0870.0871.0872.0873.0874.0875.0876.0877.0878.0879.0880.0881.0882.0883.0884.0885.0886.0887.0888.0889.0890.0891.0892.0893.0894.0895.0896.0897.0898.0899.0900.0901.0902.0903.0904.0905.0906.0907.0908.0909.0910.0911.0912.0913.0914.0915.0916.0917.0918.0919.0920.0921.0922.0923.0924.0925.0926.0927.0928.0929.0930.0931.0932.0933.0934.0935.0936.0937.0938.0939.0940.0941.0942.0943.0944.0945.0946.0947.0948.0949.0950.0951.0952.0953.0954.0955.0956.0957.0958.0959.0960.0961.0962.0963.0964.0965.0966.0967.0968.0969.0970.0971.0972.0973.0974.0975.0976.0977.0978.0979.0980.0981.0982.0983.0984.0985.0986.0987.0988.0989.0990.0991.0992.0993.0994.0995.0996.0997.0998.0999.1000.1001.1002.1003.1004.1005.1006.1007.1008.1009.1010.1011.1012.1013.1014.1015.1016.1017.1018.1019.1020.1021.1022.1023.1024.1025.1026.1027.1028.1029.1030.1031.1032.1033.1034.1035.1036.1037.1038.1039.1040.1041.1042.1043.1044.1045.1046.1047.1048.1049.1050.1051.1052.1053.1054.1055.1056.1057.1058.1059.1060.1061.1062.1063.1064.1065.1066.1067.1068.1069.1070.1071.1072.1073.1074.1075.1076.1077.1078.1079.1080.1081.1082.1083.1084.1085.1086.1087.1088.1089.1090.1091.1092.1093.1094.1095.1096.1097.1098.1099.1100.1101.1102.1103.1104.1105.1106.1107.1108.1109.1110.1111.1112.1113.1114.1115.1116.1117.1118.1119.1120.1121.1122.1123.1124.1125.1126.1127.1128.1129.1130.1131.1132.1133.1134.1135.1136.1137.1138.1139.1140.1141.1142.1143.1144.1145.1146.1147.1148.1149.1150.1151.1152.1153.1154.1155.1156.1157.1158.1159.1160.1161.1162.1163.1164.1165.1166.1167.1168.1169.1170.1171.1172.1173.1174.1175.1176.1177.1178.1179.1180.1181.1182.1183.1184.1185.1186.1187.1188.1189.1190.1191.1192.1193.1194.1195.1196.1197.1198.1199.1200.1201.1202.1203.1204.1205.1206.1207.1208.1209.1210.1211.1212.1213.1214.1215.1216.1217.1218.1219.1220.1221.1222.1223.1224.1225.1226.1227.1228.1229.1230.1231.1232.1233.1234.1235.1236.1237.1238.1239.1240.1241.1242.1243.1244.1245.1246.1247.1248.1249.1250.1251.1252.1253.1254.1255.1256.1257.1258.1259.1260.1261.1262.1263.1264.1265.1266.1267.1268.1269.1270.1271.1272.1273.1274.1275.1276.1277.1278.1279.1280.1281.1282.1283.1284.1285.1286.1287.1288.1289.1290.1291.1292.1293.1294.1295.1296.1297.1298.1299.1300.1301.1302.1303.1304.1305.1306.1307.1308.1309.1310.1311.1312.1313.1314.1315.1316.1317.1318.1319.1320.1321.1322.1323.1324.1325.1326.1327.1328.1329.1330.1331.1332.1333.1334.1335.1336.1337.1338.1339.1340.1341.1342.1343.1344.1345.1346.1347.1348.1349.1350.1351.1352.1353.1354.1355.1356.1357.1358.1359.1360.1361.1362.1363.1364.1365.1366.1367.1368.1369.1370.1371.1372.1373.1374.1375.1376.1377.1378.1379.1380.1381.1382.1383.1384.1385.1386.1387.1388.1389.1390.1391.1392.1393.1394.1395.1396.1397.1398.1399.1400.1401.1402.1403.1404.1405.1406.1407.1408.1409.1410.1411.1412.1413.1414.1415.1416.1417.1418.1419.1420.1421.1422.1423.1424.1425.1426.1427.1428.1429.1430.1431.1432.1433.1434.1435.1436.1437.1438.1439.1440.1441.1442.1443.1444.1445.1446.1447.1448.1449.1450.1451.1452.1453.1454.1455.1456.1457.1458.1459.1460.1461.1462.1463.1464.1465.1466.1467.1468.1469.1470.1471.1472.1473.1474.1475.1476.1477.1478.1479.1480.1481.1482.1483.1484.1485.1486.1487.1488.1489.1490.1491.1492.1493.1494.1495.1496.1497.1498.1499.1500.1501.1502.1503.1504.1505.1506.1507.1508.1509.1510.1511.1512.1513.1514.1515.1516.1517.1518.1519.1520.1521.1522.1523.1524.1525.1526.1527.1528.1529.1530.1531.1532.1533.1534.1535.1536.1537.1538.1539.1540.1541.1542.1543.1544.1545.1546.1547.1548.1549.1550.1551.1552.1553.1554.1555.1556.1557.1558.1559.1560.1561.1562.1563.1564.1565.1566.1567.1568.1569.1570.1571.1572.1573.1574.1575.1576.1577.1578.1579.1580.1581.1582.1583.1584.1585.1586.1587.1588.1589.1590.1591.1592.1593.1594.1595.1596.1597.1598.1599.1600.1601.1602.1603.1604.1605.1606.1607.1608.1609.1610.1611.1612.1613.1614.1615.1616.1617.1618.1619.1620.1621.1622.1623.1624.1625.1626.1627.1628.1629.1630.1631.1632.1633.1634.1635.1636.1637.1638.1639.1640.1641.1642.1643.1644.1645.1646.1647.1648.1649.1650.1651.1652.1653.1654.1655.1656.1657.1658.1659.1660.1661.1662.1663.1664.1665.1666.1667.1668.1669.1670.1671.1672.1673.1674.1675.1676.1677.1678.1679.1680.1681.1682.1683.1684.1685.1686.1687.1688.1689.1690.1691.1692.1693.1694.1695.1696.1697.1698.1699.1700.1701.1702.1703.1704.1705.1706.1707.1708.1709.1710.1711.1712.1713.1714.1715.1716.1717.1718.1719.1720.1721.1722.1723.1724.1725.1726.1727.1728.1729.1730.1731.1732.1733.1734.1735.1736.1737.1738.1739.1740.1741.1742.1743.1744.1745.1746.1747.1748.1749.1750.1751.1752.1753.1754.1755.1756.1757.1758.1759.1760.1761.1762.1763.1764.1765.1766.1767.1768.1769.1770.1771.1772.1773.1774.1775.1776.1777.1778.1779.1780.1781.1782.1783.1784.1785.1786.1787.1788.1789.1790.1791.1792.1793.1794.1795.1796.1797.1798.1799.1800.1801.1802.1803.1804.1805.1806.1807.1808.1809.1810.1811.1812.1813.1814.1815.1816.1817.1818.1819.1820.1821.1822.1823.1824.1825.1826.1827.1828.1829.1830.1831.1832.1833.1834.1835.1836.1837.1838.1839.1840.1841.1842.1843.1844.1845.1846.1847.1848.1849.1850.1851.1852.1853.1854.1855.1856.1857.1858.1859.1860.1861.1862.1863.1864.1865.1866.1867.1868.1869.1870.1871.1872.1873.1874.1875.1876.1877.1878.1879.1880.1881.1882.1883.1884.1885.1886.1887.1888.1889.1890.1891.1892.1893.1894.1895.1896.1897.1898.1899.1900.1901.1902.1903.1904.1905.1906.1907.1908.1909.1910.1911.1912.1913.1914.1915.1916.1917.1918.1919.1920.1921.1922.1923.1924.1925.1926.1927.1928.1929.1930.1931.1932.1933.1934.1935.1936.1937.1938.1939.1940.1941.1942.1943.1944.1945.1946.1947.1948.1949.1950.1951.1952.1953.1954.1955.1956.1957.1958.1959.1960.1961.1962.1963.1964.1965.1966.1967.1968.1969.1970.1971.1972.1973.1974.1975.1976.1977.1978.1979.1980.1981.1982.1983.1984.1985.1986.1987.1988.1989.1990.1991.1992.1993.1994.1995.1996.1997.1998.1999.2000.2001.2002.2003.2004.2005.2006.2007.2008.2009.2010.2011.2012.2013.2014.2015.2016.2017.2018.2019.2020.2021.2022.2023.2024.2025.2026.2027.2028.2029.2030.2031.2032.2033.2034.2035.2036.2037.2038.2039.2040.2041.2042.2043.2044.2045.2046.2047.2048.2049.2050.2051.2052.2053.2054.2055.2056.2057.2058.2059.2060.2061.2062.2063.2064.2065.2066.2067.2068.2069.2070.2071.2072.2073.2074.2075.2076.2077.2078.2079.2080.2081.2082.2083.2084.2085.2086.2087.2088.2089.2090.2091.2092.2093.2094.2095.2096.2097.2098.2099.2100.2101.2102.2103.2104.2105.2106.2107.2108.2109.2110.2111.2112.2113.2114.2115.2116.2117.2118.2119.2120.2121.2122.2123.2124.2125.2126.2127.2128.2129.2130.2131.2132.2133.2134.2135.2136.2137.2138.2139.2140.2141.2142.2143.2144.2145.2146.2147.2148.2149.2150.2151.2152.2153.2154.2155.2156.2157.2158.2159.2160.2161.2162.2163.2164.2165.2166.2167.2168.2169.2170.2171.2172.2173.2174.2175.2176.2177.2178.2179.2180.2181.2182.2183.2184.2185.2186.2187.2188.2189.2190.2191.2192.2193.2194.2195.2196.2197.2198.2199.2200.2201.2202.2203.2204.2205.2206.2207.2208.2209.2210.2211.2212.2213.2214.2215.2216.2217.2218.2219.2220.2221.2222.2223.2224.2225.2226.2227.2228.2229.2230.2231.2232.2233.2234.2235.2236.2237.2238.2239.2240.2241.2242.2243.2244.2245.2246.2247.2248.2249.2250.2251.2252.2253.2254.2255.2256.2257.2258.2259.2260.2261.2262.2263.2264.2265.2266.2267.2268.2269.2270.2271.2272.2273.2274.2275.2276.2277.2278.2279.2280.2281.2282.2283.2284.2285.2286.2287.2288.2289.2290.2291.2292.2293.2294.2295.2296.2297.2298.2299.2300.2301.2302.2303.2304.2305.2306.2307.2308.2309.2310.2311.2312.2313.2314.2315.2316.2317.2318.2319.2320.2321.2322.2323.2324.2325.2326.2327.2328.2329.2330.2331.2332.2333.2334.2335.2336.2337.2338.2339.2340.2341.2342.2343.2344.2345.2346.2347.2348.2349.2350.2351.2352.2353.2354.2355.2356.2357.2358.2359.2360.2361.2362.2363.2364.2365.2366.2367.2368.2369.2370.2371.2372.2373.2374.2375.2376.2377.2378.2379.2380.2381.2382.2383.2384.2385.2386.2387.2388.2389.2390.2391.2392.2393.2394.2395.2396.23

No.	Time	Source	Destination
11	24.042223435	10.0.0.1	124.16.77.5
12	24.043229145	124.16.77.5	10.0.0.1
13	24.043261499	10.0.0.1	124.16.77.5
14	24.055594528	10.0.0.1	124.16.77.5
15	24.056653809	124.16.77.5	10.0.0.1
16	24.067856429	124.16.77.5	10.0.0.1
17	24.067866850	10.0.0.1	124.16.77.5
18	24.069265253	10.0.0.1	124.16.77.5
19	24.071366465	124.16.77.5	10.0.0.1
20	24.071418246	10.0.0.1	124.16.77.5
21	24.081487799	124.16.77.5	10.0.0.1
22	24.081510572	10.0.0.1	124.16.77.5
23	24.082570769	124.16.77.5	10.0.0.1
24	24.082576432	10.0.0.1	124.16.77.5

▶ Frame 14: 474 bytes on wire (3792 bits), 474 bytes captured on interface
 ▶ Ethernet II, Src: ae:fa:e2:66:a3:ba (ae:fa:e2:66:a3:ba), Dst: 10.0.0.1
 ▶ Internet Protocol Version 4, Src: 10.0.0.1, Dst: 124.16.77.5
 ▶ Transmission Control Protocol, Src Port: 44642, Dst Port: 80
 ▶ Transport Layer Security

1.3.6 UCAS 页面获取过程

- 用户首先使用 `wget` 请求 `https://www.ucas.ac.cn`
- 此时客户端只知道域名，还不知道服务器的IP地址，因此首先向DNS服务器 `8.8.8.8` 发送DNS查询。DNS 返回 `www.ucas.ac.cn -> 124.16.77.5`
 - 客户端得到服务器IP地址后，需要通过网络将数据发送出去
- 在发送Ethernet帧前，h1 使用ARP查询下一跳 `10.0.0.3` 的MAC地址，并得到 `10.0.0.3 -> ce:73:a6:35:88:a5`
- h1使用本地端口 `44642` 与服务器 `124.16.77.5:443` 建立TCP连接。双方依次完成 `SYN`，`SYN,ACK`，`ACK` 三次握手
- TCP连接建立后，由于访问的是HTTPS网站，客户端继续发送TLS Client Hello，并与服务器建立加密连接
 - 随后HTTP请求及网页响应通过TLS加密，并由TCP完成后续的数据传输

2 传输性能测量实验

2.1 实验内容

本实验在Linux环境下编写python测量脚本，对约1000个网站进行网络传输性能测量。

测量的主要指标包括：

- Ping
- DNS
- TCP transfer
- Total time

同时记录TCP握手时间，用于进一步分析传输过程中不同阶段的时间开销。

在获得网站服务器 IP 地址后，使用GeoIP数据库查询服务器对应的大致地理位置，根据客户端与服务器之间的地理距离估算理论传播时延c-latency，并将实际网络传输性能与 c-latency 进行比较。

2.2 实验流程

2.2.1 测试网站选取

首先从 Tranco 网站排名数据中按排名顺序读取域名。

由于部分网站在当前网络环境下无法正常解析或访问，因此编写筛选脚本，对候选网站逐一进行DNS解析和HTTP/HTTPS可访问性测试

筛选脚本最终选取 1000 个能够正常解析和访问的网站，保存到 `sites.txt`

```
sites.txt
1  cloudflare.com
2  microsoft.com
3  amazonaws.com
4  apple.com
5  mail.ru
6  dzen.ru
7  linkedin.com
8  office.com
9  googletagmanager.com
10 live.com
11 amazon.com
12 azure.com
13 bing.com
14 appsflyersdk.com
15 fastly.net
16 netflix.com
17 sharepoint.com
18 gandi.net
19 digicert.com
20 cloud.microsoft
```

筛选脚本 `filter_sites.py` 如下：

```
1  import csv
2  import socket
3  import subprocess
4  import time
5
6  INPUT_FILE = "tranco_Q2Q44.csv"
7  OUTPUT_FILE = "sites.txt"
8
9  TARGET_COUNT = 1000
10 SITE_TIMEOUT = 8 # 设置最大测试时长
11 CONNECT_TIMEOUT = 3
12
```

```

13
14 def can_resolve(domain):
15     try:
16         socket.setdefaulttimeout(SITE_TIMEOUT)
17         socket.gethostbyname(domain)
18         return True
19     except Exception:
20         return False
21
22
23 def can_access(domain, remaining_time):
24     if remaining_time <= 0:
25         return False
26
27     schemes = ["https", "http"]
28
29     for scheme in schemes:
30         if remaining_time <= 0:
31             return False
32
33         url = f"{scheme}://{domain}"
34
35         max_time = max(1, int(remaining_time))
36
37         try:
38             result = subprocess.run(
39                 [
40                     "curl",
41                     "--noproxy", "*",
42                     "-L",
43                     "-o", "/dev/null",
44                     "-s",
45                     "--connect-timeout", str(CONNECT_TIMEOUT),
46                     "--max-time", str(max_time),
47                     url,
48                 ],
49                 stdout=subprocess.DEVNULL,
50                 stderr=subprocess.DEVNULL,
51                 timeout=remaining_time + 1,
52             )
53
54             if result.returncode == 0:
55                 return True
56
57         except subprocess.TimeoutExpired:
58             return False
59         except Exception:
60             pass
61
62         return False
63
64     return False
65

```

```

66
67 def valid_site(domain):
68     start = time.monotonic()
69
70     if not can_resolve(domain):
71         return False
72
73     elapsed = time.monotonic() - start
74     remaining = SITE_TIMEOUT - elapsed
75
76     if remaining <= 0:
77         return False
78
79     return can_access(domain, remaining)
80
81
82 def main():
83     count = 0
84     checked = 0
85
86     with open(INPUT_FILE, "r", encoding="utf-8-sig", newline="") as f, \
87         open(OUTPUT_FILE, "w", encoding="utf-8") as out:
88
89         reader = csv.reader(f)
90
91         for row in reader:
92             if len(row) < 2:
93                 continue
94
95             domain = row[1].strip()
96
97             if not domain:
98                 continue
99
100             checked += 1
101
102             print(
103                 f"[checked={checked}, valid={count}/{TARGET_COUNT}] "
104                 f"testing {domain} ..."
105             )
106
107             start = time.monotonic()
108             ok = valid_site(domain)
109             elapsed = time.monotonic() - start
110
111             if ok:
112                 count += 1
113                 out.write(domain + "\n")
114                 out.flush()
115
116             print(
117                 f" OK ({elapsed:.2f}s) "
118                 f"-> saved as #{count}"

```

```

119         )
120     else:
121         print(
122             f"    FAIL ({elapsed:.2f}s)"
123         )
124
125     if count >= TARGET_COUNT:
126         break
127
128     print()
129     print(f"Checked sites: {checked}")
130     print(f"Valid sites:    {count}")
131     print(f"Output:          {OUTPUT_FILE}")
132
133
134 if __name__ == "__main__":
135     main()
136

```

2.2.2 网络性能测量

使用python编写 `measure.py`，逐个读取 `sites.txt` 中的网站进行测量。

对于每个网站，首先使用 DNS 解析获得服务器 IPv4 地址；然后通过 `ping` 测量网络 RTT；同时使用 `curl` 获取 DNS 解析、TCP 建连和整个页面传输过程的时间信息。

程序最终记录以下字段：

```

1  rank
2  domain
3  ip
4  ping_ms
5  dns_ms
6  tcp_handshake_ms
7  tcp_transfer_ms
8  total_ms
9  url

```

其中：

- `DNS time = time_namelookup`
- TCP三次握手时间计算为 `TCP handshake = time_connect - time_namelookup`
- TCP transfer计算为 `TCP transfer = time_total - time_connect`
- 总传输时间为 `Total time = time_total`

测量结果保存到 `results.csv`

```

results.csv > data
1 rank,domain,ip,ping_ms,dns_ms,tcp_handshake_ms,tcp_transfer_ms,total_ms,url
2 1,cloudflare.com,104.16.133.229,216.476,17.855,393.0469999999997,11305.25,11716.152,https://cloudflare.com
3 2,microsoft.com,150.171.110.70,,27.743,246.51000000000005,3372.335,3646.5879999999997,https://microsoft.com
4 3,amazonaws.com,15.197.250.250,78.459,20.159,278.141,5922.28,6220.58,http://amazonaws.com
5 4,apple.com,17.253.144.10,98.987,30.86200000000002,67.826,718.8149999999999,817.5029999999999,https://apple.com
6 5,mail.ru,185.180.201.1,235.765,560.838,597.4509999999999,1429.4800000000002,2587.7690000000002,https://mail.ru
7 6,dzen.ru,95.163.218.220,,1044.096,389.6260000000003,1033.9930000000002,2467.715,https://dzen.ru
8 7,linkedin.com,52.131.37.152,11.449,56.50199999999995,34.73900000000004,2699.185,2790.426,https://linkedin.com
9 8,office.com,13.107.6.156,73.008,50.33,204.784,1653.3200000000002,1998.434,https://office.com
10 9,googletagmanager.com,220.,Col 4: ping_ms,2,1027.811,17.2539999999998,1137.2839999999999,2182.348999999997,https://
11 10,live.com,204.79.197.212,87.529,34.389,224.976,1404.262,1663.627,https://live.com
12 11,amazon.com,98.87.170.71,,17.058,331.9439999999996,881.837,1230.839,https://amazon.com
13 12,azure.com,20.76.201.171,287.052,1433.192,284.2229999999999,5678.71,7396.125,https://azure.com
14 13,bing.com,150.171.27.10,84.707,38.3,499.339,956.5920000000001,1494.231,https://bing.com
15 14,appsflyersdk.com,65.9.168.12,93.896,56.583,101.77,179.162,337.515,https://appsflyersdk.com
16 15,fastly.net,151.101.65.6,185.543,349.967,264.9880000000006,8213.378999999999,8828.334,https://fastly.net
17 16,netflix.com,44.242.13.161,,21.29800000000002,184.6039999999998,426.3699999999999,632.2719999999999,https://netfli
18 17,sharepoint.com,13.107.136.10,59.285,151.881,717.157,2803.394,3672.4320000000002,https://sharepoint.com
19 18,gandi.net,217.70.185.65,177.623,336.911,340.0850000000004,1637.396,2314.392,https://gandi.net
20 19,digicert.com,45.60.121.229,152.995,148.208,480.033,1287.629,1915.87,https://digicert.com
21 20,cloud.microsoft,20.76.201.171,252.945,38.031,251.15300000000002,577.851,867.035,https://cloud.microsoft
22 21,wordpress.org,66.6.42.252,229.183,19.227,274.132,1560.315,1853.674,https://wordpress.org
23 22,skype.com,20.112.250.133,139.181,38.382,376.6,1636.48,2051.462,https://skype.com
24 23,icloud.com,17.253.144.10,84.73,28.014,184.2839999999996,550.923,763.221,https://icloud.com
25 24,cloudflare.net,104.16.208.90,186.487,15.95200000000002,163.072,341.1850000000006,520.2090000000001,https://cloudf
26 25,roblox.com,128.116.31.3,233.632,186.9609999999998,483.4029999999996,1263.973,1934.337,https://roblox.com
27 26,yahoo.com,98.137.11.164,189.018,28.055,387.545,782.173,1197.773,https://yahoo.com
28 27,ntp.org,204.93.207.22,271.378,18.227,399.553,1431.1829999999998,1848.963,https://ntp.org
29 28,msn.com,204.79.197.219,89.496,38.119,463.8639999999999,1339.169,1841.1519999999998,https://msn.com
30 29,adobe.com,23.2.16.113,274.597,20.65799999999998,444.2979999999994,5238.044000000001,5703.0,https://adobe.com

```

本次共测试 1000 个网站，得到的有效结果如下：

- 1 测试网站总数：1000
- 2 IP 解析成功：1000
- 3 Ping 测量成功：824
- 4 HTTP 测量成功：992

measure.py 如下：

```

1 import csv
2 import re
3 import socket
4 import subprocess
5 import time
6
7 INPUT_FILE = "sites.txt"
8 OUTPUT_FILE = "results.csv"
9
10 PING_COUNT = 3
11 TIMEOUT = 10
12
13
14 def resolve_ip(domain):
15     try:
16         return socket.gethostbyname(domain)
17     except Exception:
18         return ""
19
20
21 def measure_ping(domain):

```

```

22     try:
23         result = subprocess.run(
24             ["ping", "-c", str(PING_COUNT), "-w", "2", domain],
25             stdout=subprocess.PIPE,
26             stderr=subprocess.DEVNULL,
27             text=True,
28             timeout=10,
29         )
30
31         m = re.search(
32             r"rtt min/avg/max/mdev = [\d.]+/([\d.]+)/",
33             result.stdout
34         )
35
36         if m:
37             return float(m.group(1))
38
39     except Exception:
40         pass
41
42     return None
43
44
45 def measure_curl(domain):
46     fmt = (
47         "%{time_name}lookup},"
48         "%{time_name}connect},"
49         "%{time_name}total}"
50     )
51
52     urls = [
53         f"https://{domain}",
54         f"http://{domain}",
55     ]
56
57     for url in urls:
58         try:
59             result = subprocess.run(
60                 [
61                     "curl",
62                     "--noproxy", "*",
63                     "-L",
64                     "-o", "/dev/null",
65                     "-s",
66                     "--connect-timeout", str(TIMEOUT),
67                     "--max-time", "20",
68                     "-w", fmt,
69                     url,
70                 ],
71                 stdout=subprocess.PIPE,
72                 stderr=subprocess.DEVNULL,
73                 text=True,
74                 timeout=25,

```

```

75         )
76
77         if result.returncode != 0:
78             continue
79
80         values = result.stdout.strip().split(",")
81
82         if len(values) != 3:
83             continue
84
85         dns = float(values[0])
86         connect = float(values[1])
87         total = float(values[2])
88
89         if total <= 0:
90             continue
91
92         dns_ms = dns * 1000
93         tcp_handshake_ms = max(0, (connect - dns) * 1000)
94         tcp_transfer_ms = max(0, (total - connect) * 1000)
95         total_ms = total * 1000
96
97         return (
98             dns_ms,
99             tcp_handshake_ms,
100            tcp_transfer_ms,
101            total_ms,
102            url,
103        )
104
105     except Exception:
106         continue
107
108     return None
109
110
111 def main():
112     with open(INPUT_FILE, "r", encoding="utf-8") as f:
113         domains = [
114             line.strip()
115             for line in f
116             if line.strip()
117         ]
118
119     with open(
120         OUTPUT_FILE,
121         "w",
122         newline="",
123         encoding="utf-8"
124     ) as f:
125
126         writer = csv.writer(f)
127

```

```

128     writer.writerow([
129         "rank",
130         "domain",
131         "ip",
132         "ping_ms",
133         "dns_ms",
134         "tcp_handshake_ms",
135         "tcp_transfer_ms",
136         "total_ms",
137         "url",
138     ])
139
140     for rank, domain in enumerate(domains, 1):
141         print(f"[{rank}/{len(domains)}] {domain}")
142
143         ip = resolve_ip(domain)
144         ping_ms = measure_ping(domain)
145         curl_result = measure_curl(domain)
146
147         if curl_result:
148             (
149                 dns_ms,
150                 handshake_ms,
151                 transfer_ms,
152                 total_ms,
153                 url,
154             ) = curl_result
155         else:
156             dns_ms = None
157             handshake_ms = None
158             transfer_ms = None
159             total_ms = None
160             url = ""
161
162         writer.writerow([
163             rank,
164             domain,
165             ip,
166             ping_ms,
167             dns_ms,
168             handshake_ms,
169             transfer_ms,
170             total_ms,
171             url,
172         ])
173
174         f.flush()
175
176         time.sleep(0.1)
177
178
179 if __name__ == "__main__":
180     main()

```

2.2.3 GeoIP获取和c-latency计算

2.2.3.1 服务器位置获取

使用DB-IP City Lite GeoIP数据库，根据 `results.csv` 中得到的服务器 IP 地址查询对应的大致地理位置，包括：

```
1 country
2 city
3 latitude
4 longitude
```

客户端位置按照实验所在的北京市怀柔区进行近似，经纬度设置为：

```
1 Latitude = 40.3160
2 Longitude = 116.6318
```

python脚本根据上述设置来计算距离，并将结果保存至 `results_geo.csv`

```
results_geo.csv > data
1 rank,domain,ip,ping_ms,dns_ms,tcp_handshake_ms,tcp_transfer_ms,total_ms,url,server_lat,server_lon,country,city,distance
2 1,cloudflare.com,104.16.133.229,216.476,17.855,393.047,11305.25,11716.152,https://cloudflare.com,43.6532,-79.3832,Canada,Toronto
3 2,microsoft.com,150.171.110.70,27.743,246.51000000000005,3372.335,3646.588,https://microsoft.com,32.7767,-96.797,United States,Capitol Hill
4 3,amazonaws.com,15.197.250.250,78.459,20.159,278.141,5922.28,6220.58,http://amazonaws.com,45.5019,-73.5674,Canada,Montreal
5 4,apple.com,17.253.144.10,98.987,30.862,67.826,718.8149999999999,817.5029999999999,https://apple.com,37.3322,-122.011,United States,Cupertino
6 5,mail.ru,185.180.201.1,235.765,560.838,597.4509999999999,1429.4800000000002,2587.769,https://mail.ru,55.7951,37.5418,Russia,Moscow
7 6,dzen.ru,95.163.218.220,1044.096,389.626,1033.9930000000002,2467.715,https://dzen.ru,55.7558,37.6173,Russia,Moscow
8 7,linkedin.com,52.131.37.152,11.449,56.502,34.739000000000004,2699.185,2790.426,https://linkedin.com,39.9042,116.407,United States,Ashburn
9 8,office.com,13.107.6.156,73.008,50.33,294.784,1653.3200000000002,1998.434,https://office.com,40.7357,-74.1724,United States,Redmond
10 9,googletagmanager.com,220.181.174.169,31.62,1027.811,17.25399999999998,1137.284,2182.349,https://googletagmanager.com,40.7128,-74.006,United States,New York
11 10,live.com,204.79.197.212,87.529,34.389,224.976,1404.262,1663.627,https://live.com,40.8618,-74.0587,United States,Tetonia
12 11,amazon.com,98.87.170.71,17.058,331.944,881.837,1230.839,https://amazon.com,39.0438,-77.4874,United States,Ashburn
13 12,azure.com,20.76.201.171,287.052,1433.192,284.22299999999999,5678.71,7396.125,https://azure.com,52.3676,4.90414,The Netherlands,Amsterdam
14 13,bing.com,150.171.27.10,84.707,38.3,499.339,956.592,1494.231,https://bing.com,40.7128,-74.006,United States,New York
15 14,appsflyersdk.com,65.9.168.12,93.896,56.583,101.77,179.162,337.515,https://appsflyersdk.com,1.35208,103.82,Singapore,Singapore
16 15,fastly.net,151.101.65.6,185.543,349.967,264.98800000000006,8213.378999999999,8828.334,https://fastly.net,45.5019,-77.0363,United States,New York
17 16,netflix.com,44.242.13.161,21.298,184.604,426.36999999999999,632.2719999999999,https://netflix.com,45.5152,-122.678,United States,Los Angeles
18 17,sharepoint.com,13.107.136.10,59.285,151.881,717.157,2803.394,3672.432,https://sharepoint.com,40.7128,-74.006,United States,New York
19 18,gandi.net,217.70.185.65,177.623,336.911,340.08500000000004,1637.396,2314.392,https://gandi.net,48.8219,2.3709,France,Paris
20 19,digicert.com,45.60.121.229,152.995,148.208,480.033,1287.629,1915.87,https://digicert.com,40.7128,-74.006,United States,New York
21 20,cloud.microsoft,20.76.201.171,252.945,38.031,251.153,577.851,867.035,https://cloud.microsoft,52.3676,4.90414,The Netherlands,Amsterdam
22 21,wordpress.org,66.6.42.252,229.183,19.227,274.132,1560.315,1853.674,https://wordpress.org,39.0438,-77.4874,United States,Ashburn
23 22,skype.com,20.112.250.133,139.181,38.382,376.6,1636.48,2051.462,https://skype.com,41.5868,-93.625,United States,Des Moines
24 23,icloud.com,17.253.144.10,84.73,28.014,184.284,550.923,763.221,https://icloud.com,37.3322,-122.011,United States,Cupertino
25 24,cloudflare.net,104.16.208.90,186.487,15.952000000000002,163.072,341.18500000000006,520.2090000000001,https://cloudflare.com,43.6532,-79.3832,Canada,Toronto
26 25,roblox.com,128.116.31.3,233.632,186.961,483.403,1263.973,1934.337,https://roblox.com,51.5072,-0.127586,United Kingdom,London
27 26,yahoo.com,98.137.11.164,189.018,28.055,387.545,782.173,1197.773,https://yahoo.com,40.7309,-73.9918,United States,New York
28 27,ntp.org,204.93.207.22,271.378,18.227,399.553,1431.1829999999998,1848.963,https://ntp.org,41.8781,-87.6298,United States,Chicago
29 28,msn.com,204.79.197.219,89.496,38.119,463.8639999999999,1339.169,1841.152,https://msn.com,40.8618,-74.0587,United States,Redmond
30 29,adobe.com,23.2.16.113,274.597,20.658,444.298,5238.044000000001,5703.0,https://adobe.com,22.3186,114.18,Hong Kong,Hong Kong
31 30,nginx.org,3.125.197.172,167.412,10.792,190.449,564.364,765.605,https://nginx.org,50.1109,8.68213,Germany,Frankfurt
32 31,zoom.us,170.114.52.2,180.118,19.947,377.538,1285.381,1682.866,https://zoom.us,37.3329,-121.895,United States,San Jose
33 32,qq.com,123.150.76.218,42.936,45.319,20.812,65.391,131.522,https://qq.com,39.0851,117.199,China,Youyilu,145.21634344
34 33,baidu.com,110.242.74.102,15.766,42.684,18.553,73.254,134.491,https://baidu.com,40.9515,117.963,China,Chengde,132.70
35 34,openai.com,104.18.33.45,188.452,16.527,213.479,465.795,695.801,https://openai.com,43.6532,-79.3832,Canada,Toronto,1
36 35,yandex.net,213.180.204.242,257.963,15.278,486.365,1303.5290000000002,1805.172,https://yandex.net,55.7342,37.5859,Russia,Moscow
37 36,windows.com,20.70.246.20,196.02,128.539,757.085,3879.967,4765.590999999999,https://windows.com,-33.8688,151.209,Australia,Sydney
```

2.2.3.2 c-latency计算

课件中给出的光速网络传播时延模型为：
$$L = \frac{D}{0.6C}$$

其中， D 为传播距离， C 为光速，光纤中的传播速度近似取 $0.6C$ 。

由于本实验中的 Ping 测量值为往返时延 RTT，因此使用往返距离计算对应的 c-latency: $c\text{-latency} = \frac{2D}{0.6C}$

数据处理脚本 geo.py 如下:

```
1 import math
2 import pandas as pd
3 import geoip2.database
4
5 INPUT_FILE = "results.csv"
6 OUTPUT_FILE = "results_geo.csv"
7 DB_FILE = "dbip-city-lite-2026-09.mmdb"
8
9 CLIENT_LAT = 40.3160
10 CLIENT_LON = 116.6318
11
12
13 def haversine(lat1, lon1, lat2, lon2):
14     R = 6371.0
15
16     phi1 = math.radians(lat1)
17     phi2 = math.radians(lat2)
18
19     dphi = math.radians(lat2 - lat1)
20     dlamba = math.radians(lon2 - lon1)
21
22     a = (
23         math.sin(dphi / 2) ** 2
24         + math.cos(phi1)
25         * math.cos(phi2)
26         * math.sin(dlamba / 2) ** 2
27     )
28
29     c = 2 * math.atan2(
30         math.sqrt(a),
31         math.sqrt(1 - a)
32     )
33
34     return R * c
35
36
37 def c_latency_rtt_ms(distance_km):
38     speed_km_s = 0.6 * 300000
39     return (
40         2 * distance_km / speed_km_s
41     ) * 1000
42
43
44 df = pd.read_csv(INPUT_FILE)
45
46 reader = geoip2.database.Reader(DB_FILE)
47
48 server_latitudes = []
49 server_longitudes = []
```

```

50 countries = []
51 cities = []
52 distances = []
53 c_latencies = []
54
55 for ip in df["ip"]:
56
57     try:
58         response = reader.city(ip)
59
60         lat = response.location.latitude
61         lon = response.location.longitude
62
63         country = response.country.name
64         city = response.city.name
65
66         if lat is None or lon is None:
67             raise ValueError()
68
69         distance = haversine(
70             CLIENT_LAT,
71             CLIENT_LON,
72             lat,
73             lon
74         )
75
76         c_latency = c_latency_rtt_ms(distance)
77
78     except Exception:
79         lat = None
80         lon = None
81         country = None
82         city = None
83         distance = None
84         c_latency = None
85
86     server_latitudes.append(lat)
87     server_longitudes.append(lon)
88     countries.append(country)
89     cities.append(city)
90     distances.append(distance)
91     c_latencies.append(c_latency)
92
93 reader.close()
94
95 df["server_lat"] = server_latitudes
96 df["server_lon"] = server_longitudes
97 df["country"] = countries
98 df["city"] = cities
99 df["distance_km"] = distances
100 df["c_latency_ms"] = c_latencies
101
102 df.to_csv(

```

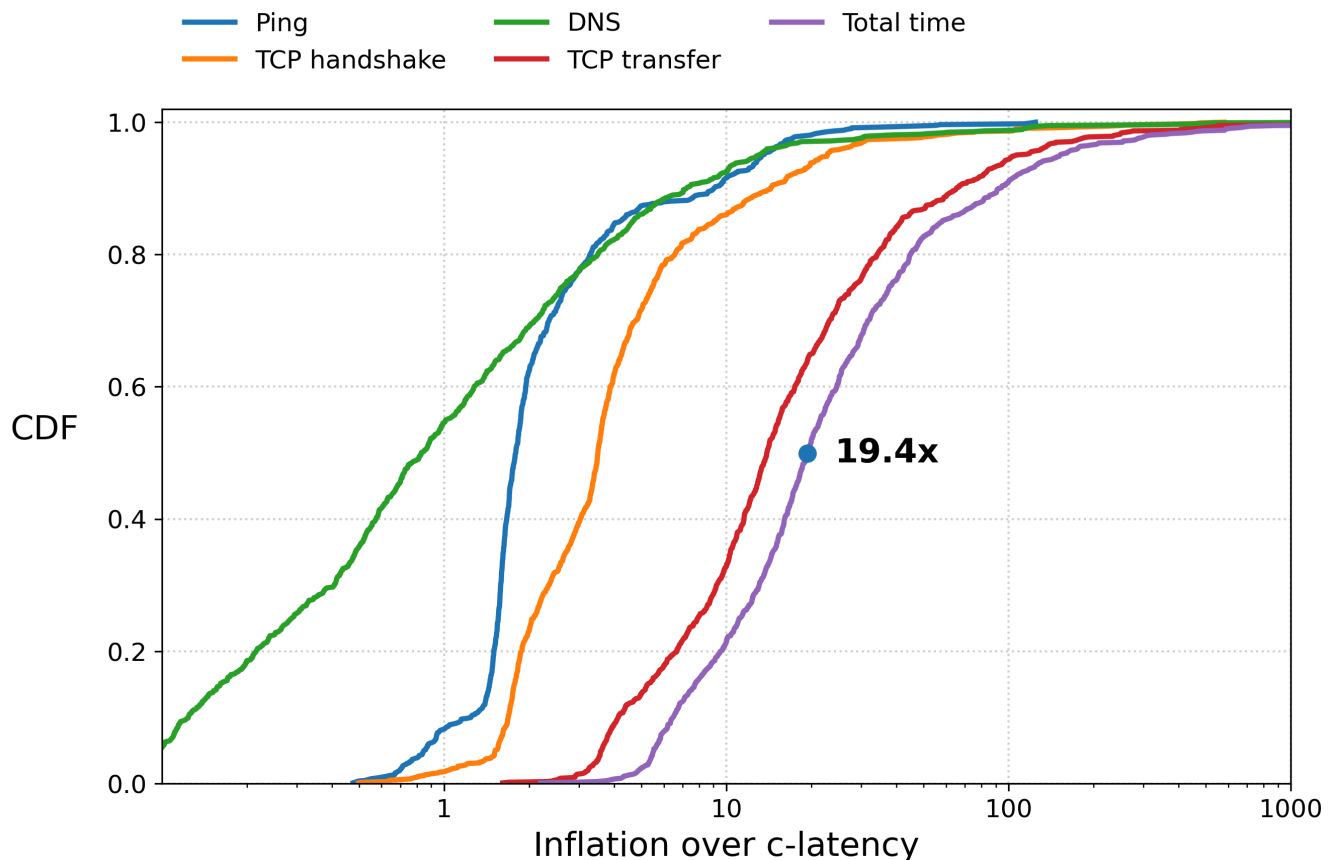
```

103     OUTPUT_FILE,
104     index=False
105 )
106
107 print(f"Saved to {OUTPUT_FILE}")

```

2.3 实验结果及分析

将各项实际测量时间除以 c-latency，得到相对理论传播时延的膨胀倍数，并绘制 CDF 曲线。



从图中可以看出，DNS和Ping的膨胀倍数整体较小，TCP handshake次之，而TCP transfer和Total time明显更大。其中Total time的中位膨胀倍数约为19.4x，说明实际完整传输时间通常远高于仅考虑传播距离得到的理论时延。

产生这一差异的原因主要包括实际网络路径长于地理直线距离，以及发送延迟、队列延迟和TCP建连等额外开销。

此外，部分样本出现实际Ping小于估算c-latency的情况，这主要是由于GeoIP只能粗略估计服务器位置，而CDN、云服务等机制可能使实际响应节点与数据库记录位置不一致。

绘图脚本 plot.py 如下：

```

1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  INPUT_FILE = "results_geo.csv"
6  OUTPUT_FILE = "cdf_inflation.png"
7
8  DRAW_ROUTER_PATH = False
9

```

```

10 def prepare_ratio(df, metric, c_col="c_latency_ms"):
11     data = df[[metric, c_col]].dropna().copy()
12
13     data = data[
14         (data[metric] > 0) &
15         (data[c_col] > 0)
16     ]
17
18     ratio = data[metric] / data[c_col]
19
20     ratio = ratio.replace([np.inf, -np.inf], np.nan).dropna()
21
22     return ratio.values
23
24
25 def ecdf(values):
26     x = np.sort(values)
27     y = np.arange(1, len(x) + 1) / len(x)
28     return x, y
29
30 def add_curve(ax, values, label, linewidth=2.5):
31     if len(values) == 0:
32         print(f"warning: no valid data for {label}")
33         return None
34
35     x, y = ecdf(values)
36
37     line, = ax.plot(
38         x,
39         y,
40         linewidth=linewidth,
41         label=label
42     )
43
44     return line
45
46
47 def main():
48     df = pd.read_csv(INPUT_FILE)
49
50     required = [
51         "ping_ms",
52         "dns_ms",
53         "tcp_handshake_ms",
54         "tcp_transfer_ms",
55         "total_ms",
56         "c_latency_ms",
57     ]
58
59     missing = [c for c in required if c not in df.columns]
60
61     if missing:
62         raise ValueError(

```

```

63         f"Missing columns in {INPUT_FILE}: {missing}"
64     )
65
66     ping_ratio = prepare_ratio(df, "ping_ms")
67     dns_ratio = prepare_ratio(df, "dns_ms")
68     handshake_ratio = prepare_ratio(df, "tcp_handshake_ms")
69     transfer_ratio = prepare_ratio(df, "tcp_transfer_ms")
70     total_ratio = prepare_ratio(df, "total_ms")
71
72     print("Valid samples:")
73     print("Min Ping:", len(ping_ratio))
74     print("DNS:", len(dns_ratio))
75     print("TCP handshake:", len(handshake_ratio))
76     print("TCP transfer:", len(transfer_ratio))
77     print("Total time:", len(total_ratio))
78
79     router_ratio = None
80
81     if DRAW_ROUTER_PATH:
82         if "router_path_ms" in df.columns:
83             router_ratio = prepare_ratio(df, "router_path_ms")
84         else:
85             print(
86                 "Warning: DRAW_ROUTER_PATH=True, "
87                 "but router_path_ms not found."
88             )
89
90     fig, ax = plt.subplots(figsize=(9, 6))
91
92     lines = []
93
94     if router_ratio is not None:
95         line = add_curve(
96             ax,
97             router_ratio,
98             "Router-path"
99         )
100         if line:
101             lines.append(line)
102
103     for values, label in [
104         (ping_ratio, "Ping"),
105         (handshake_ratio, "TCP handshake"),
106         (dns_ratio, "DNS"),
107         (transfer_ratio, "TCP transfer"),
108         (total_ratio, "Total time"),
109     ]:
110         line = add_curve(
111             ax,
112             values,
113             label
114         )
115

```

```

116         if line:
117             lines.append(line)
118
119     ax.set_xscale("log")
120
121     ax.set_xlim(0.1, 1000)
122     ax.set_ylim(0, 1.02)
123
124     ax.set_xlabel(
125         "Inflation over c-latency",
126         fontsize=16
127     )
128
129     ax.set_ylabel(
130         "CDF",
131         fontsize=16,
132         rotation=0,
133         labelpad=30
134     )
135
136     ax.grid(
137         True,
138         which="major",
139         linestyle=":",
140         linewidth=1,
141         alpha=0.6
142     )
143
144     ax.set_yticks(
145         np.arange(0, 1.01, 0.2)
146     )
147
148     ax.set_xticks(
149         [1, 10, 100, 1000]
150     )
151
152     ax.get_xaxis().set_major_formatter(
153         plt.ScalarFormatter()
154     )
155
156     ax.tick_params(
157         axis="both",
158         labelsize=12
159     )
160
161     ax.legend(
162         loc="lower left",
163         bbox_to_anchor=(0.0, 1.02),
164         ncol=3,
165         frameon=False,
166         fontsize=12
167     )
168

```

```

169     if len(total_ratio) > 0:
170         median_total = np.median(total_ratio)
171
172         x_total, y_total = ecdf(total_ratio)
173
174         idx = np.argmin(
175             np.abs(y_total - 0.5)
176         )
177
178         x50 = x_total[idx]
179         y50 = y_total[idx]
180
181         ax.scatter(
182             [x50],
183             [y50],
184             s=60,
185             zorder=5
186         )
187
188         ax.text(
189             x50 * 1.25,
190             y50,
191             f"{median_total:.1f}x",
192             fontsize=16,
193             fontweight="bold",
194             va="center"
195         )
196
197     plt.tight_layout()
198
199     plt.savefig(
200         OUTPUT_FILE,
201         dpi=300,
202         bbox_inches="tight"
203     )
204
205     print(f"saved figure to {OUTPUT_FILE}")
206
207     plt.show()
208
209
210 if __name__ == "__main__":
211     main()

```

